

## Práctica 2: Interacción no verbal

### Objetivo

El objetivo de esta práctica es adquirir experiencia con los métodos de interacción no verbal en la comunicación persona-robot, particularmente aquellos relacionados con el sentido de la visión. Para eso vas a explorar algunos algoritmos simples que permitan la detección de la expresión facial de la persona (posición de cara y ojos), o postura (manos), y combinarlos para construir un sistema conversacional muy básico.

### Enunciado

Una empresa dedicada a la educación a distancia (vía telemática) ha descubierto que los alumnos se involucran más en las lecciones cuando el narrador del tema (vídeo/apuntes) no es sólo una voz en off, sino que interactúa un poco con el alumno. Por ejemplo, si el alumno no está mirando la pantalla, detiene la narración hasta que vuelva a prestarle atención. O lo anima si lo encuentra algo despistado.

Eres contratado para implementar un prototipo de esta **plataforma de gestión de la atención**. Con las librerías adecuadas, la cámara web del ordenador del alumno<sup>1</sup> puede identificar en tiempo real:

- si la persona está mirando la pantalla (*face detection*)
- si los ojos están abiertos y mirando a la pantalla (*eye tracking*)
- si levanta la mano (*posture detection*)
- qué emoción expresa la persona (*emotion detection*: ira, felicidad, sorpresa, miedo... o simple expresión neutral)

Esta información la vas a utilizar para construir un *gestor de diálogos interactivo* con la intención de motivar al alumno a prestar atención.

Ejemplos de "diálogo":

#### Ejemplo 1

```
TUTOR VIRTUAL: Blah, blah ... blah.
(el ALUMNO deja de mirar directamente a la pantalla, bajando la cabeza para
revisar el móvil)
(el TUTOR VIRTUAL se detiene)
(el ALUMNO sigue sin mirar directamente a la pantalla)
TUTOR VIRTUAL: (beep) (beep) ¿Qué? ¿Continuamos?
(el ALUMNO vuelve a mirar a la pantalla)
TUTOR VIRTUAL: Como iba diciendo...
TUTOR VIRTUAL: Blah, blah ... blah.
```

---

<sup>1</sup>En caso de que las prácticas se realicen en un ordenador sin cámara web, se puede utilizar un vídeo como datos de entrada; los cambios en el código son mínimos.

## Ejemplo 2

TUTOR VIRTUAL: Blah, blah ... blah.  
(el ALUMNO cierra los ojos)  
TUTOR VIRTUAL: Blah, blah ... blah.  
(el ALUMNO sigue con los ojos cerrados)  
(el TUTOR VIRTUAL se detiene)  
(el ALUMNO sigue con los ojos cerrados)  
TUTOR VIRTUAL: (beep) (beep) ¿Qué? ¿Continuamos?  
(el ALUMNO sigue con los ojos cerrados)  
TUTOR VIRTUAL: (beep) (beep) ¿Esto te aburre?  
(el ALUMNO sigue con los ojos cerrados)  
TUTOR VIRTUAL: Pues lo dejamos aquí. Ya continuaremos en otro momento.  
(el sistema termina)

## Ejemplo 3

TUTOR VIRTUAL: Blah, blah ... blah.  
TUTOR VIRTUAL: (cuenta un chiste)  
(el ALUMNO sonríe; eso es señal de que está siguiendo la explicación: puntos extra para él)

## Ejemplo 4

TUTOR VIRTUAL: Blah, blah ... blah.  
(el ALUMNO levanta la mano)  
TUTOR VIRTUAL: ¿Qué? ¿Quieres un descanso?  
(el TUTOR VIRTUAL espera sin quejarse)  
(el ALUMNO levanta de nuevo la mano)  
TUTOR VIRTUAL: Como iba diciendo...  
TUTOR VIRTUAL: Blah, blah ... blah.

## Implementación

En esta práctica vas a construir y explorar varios algoritmos sencillos (*face detection*, *eyes tracking*, *posture detection* y *emotion detection*), y utilizarlos para hacer avanzar el "diálogo" (unidireccional, sólo la máquina va a hablar; el alumno, como mucho, cambia de expresión). Únelos para construir el prototipo del tutor virtual.

No hará falta implementar desde cero los algoritmos necesarios, sino que van a estar contruidos sobre librerías oficiales del campo de visión artificial: *mediapipe* y *deepface* (alternativamente, *FER*). Como la documentación disponible no es muy extensa, se proporcionan algunos ejemplos sencillos de uso. En ocasiones se va a utilizar además *opencv* (y a lo mejor *numpy*). Aquí la documentación es suficiente y no deberías tener problemas.

La **idea básica** es que si la persona presta atención, la lección avanza; si sólo presta atención a ratos, la máquina reacciona con frases de apoyo ("venga, que no es tan complicado", "¿qué? ¿continuamos un poco más?") o deteniéndose; y si presta muy poca atención, a la máquina se le acaba la paciencia y termina de forma brusca ("pues seguimos en otro momento").

El sistema puede implementarse como una aplicación donde el programa lee de un fichero de texto las frases una a una y las muestra en pantalla (durante uno o dos segundos cada frase, por ejemplo); alternatively puede mostrar simplemente frases como "*Blah, blah... blah*". Periódicamente (cada dos segundos, por ejemplo) revisa la cámara web, extrae una imagen, calcula la atención del usuario a través de tus algoritmos y actúa en consecuencia:

Si el alumno atiende, no pasa nada.

En caso contrario, puede detener la lectura, mostrar distintas frases, toser, pitar (*beep*) (*beep*), llamarlo por su nombre o detenerse furioso como última opción.

Una posibilidad es probar también que está atendiendo realmente y no soñando con los ojos abiertos; el sistema puede contar anécdotas o chistes; si el usuario está prestando atención, su expresión facial debe cambiar...

## Algoritmos

**Subsistema de detección de caras.** La librería *mediapipe* contiene varios modelos para detectar caras que identifican en la imagen seis puntos críticos:

- Posición del ojo derecho; Posición del ojo izquierdo; Posición de la nariz; Posición de la boca; Posición de la oreja derecha; Posición de la oreja izquierda

Utiliza esta información (cada elemento está localizado en su punto medio por las coordenadas relativas  $(x, y)$ ) para identificar la posición aproximada de la cabeza.

La idea que puedes explotar es que cuando se mira al frente, la posición de boca y nariz corresponden más o menos con la con la posición central entre los dos ojos. Pero sólo si la cabeza NO ESTÁ inclinada (es decir, si ambos ojos comparten aproximadamente la misma altura). Si la cabeza estuviera inclinada no sería mucho más difícil de calcular (son cálculos geométricos sencillos), pero ya que estamos construyendo un prototipo podemos simplemente avisar de que el usuario está mal posicionado e ignorar el cálculo.

Revisa los ejemplos para ver cómo acceder a esta información.

**Subsistema de detección de la mirada.** La librería *mediapipe* incluye múltiples capacidades para identificar objetos en imágenes, muchas de ellas relacionadas con el cuerpo humano. Ya incluye un método para calcular la posición del iris de los ojos en la imagen e incluso calcular a partir de su tamaño la distancia a la cámara. Si lo encuentras útil, perfecto. Alternativamente, se puede utilizar la aproximación clásica:

- para identificar la posición del iris en cada ojo, se divide el ojo horizontalmente en tres partes iguales e identificar cuál de ellas tiene mayor número de píxeles negros; si es la primera parte, el ojo está mirando a la derecha; si es la segunda, mira al frente; si es la tercera, mira a la izquierda; si ninguna de ellas es predominante, el ojo está cerrado.

Revisa los ejemplos para ver cómo acceder a esta información.

**Subsistema de detección de postura/manos.** No hace falta hacer una detección de postura completa. Se puede limitar a un detector de manos.

Revisa los ejemplos para ver cómo acceder a esta información.

**Subsistema de detección de la emoción.** Hay un par de librerías que se usan para esta tarea, *deepface* y *FER*. La primera funciona mejor cuando el equipo no dispone de una GPU y será la elección preferida. Ambas librerías clasifican la expresión facial en una de seis posibilidades (cada una con su puntuación); se pueden mostrar todas las puntuaciones o sólo la emoción principal. En el caso de *deepface*, además están disponibles otros identificadores para edad, género y etnia.

De nuevo, revisa los ejemplos.

## Extras

Idealmente versiones posteriores del prototipo podrían introducir otros signos y señales (no lo hagas):

Por ejemplo, el reconocimiento de cabeceo vertical u horizontal (para afirmar algo o negarlo); no se contemplan en este prototipo porque se toman imágenes de la cámara con poca frecuencia (por ejemplo, una imagen cada segundo) que no es suficiente para reconocer este gesto (que tarda menos de un segundo en una conversación típica). O utilizar sonidos típicos (toses, por ejemplo) para llamar la atención del alumno despistado. O complementar el tutor dibujándolo en pantalla. O...

## Entrega

La práctica está pensada para ser realizada en equipos (especificad quiénes en la propia memoria). El tiempo de dedicación será de tres sesiones de prácticas, al principio del curso. En caso de no ser suficiente, completadla en vuestro tiempo libre *SIN PASARSE*. El plazo máximo de entrega son 2 semanas después de la última sesión de la práctica. La entrega se realizará a través del campus virtual (fichero ZIP; es suficiente con que una persona del equipo lo entregue):

- La memoria en PDF.
- Los códigos python resultado.
- Un vídeo breve (*screencast*) que demuestre que funciona.

Después de entregarla contactad conmigo para presentármela (de forma presencial o desde *Teams*) en cinco/diez minutos, y que pueda puntuarla.

## Memoria

La memoria se entregará en formato PDF con un máximo de 15 páginas en total:

1. Descripción del diseño e implementación de la práctica: Qué apartados se han implementado. Qué algoritmos se proponen. Simplificaciones y/o limitaciones en vuestra implementación. Otras cuestiones relevantes que queráis comentar:

¿se detecta la cabeza cuando está inclinada? ¿cuánto podemos girar la cabeza antes de perder la posición de los ojos? ¿detecta correctamente el caso de ojos cerrados? ¿y si hay gafas? ¿qué pasa si el usuario guiña el ojo? ¿si estornuda cambian los resultados? ¿cuánto influye la luz ambiente en la detección? ¿qué pasa si hay varios usuarios en el foco de la cámara?

2. Análisis de resultados. ¿Cuáles son los retos principales encontrados en cada apartado? ¿Posibles mejoras, si hubiera tiempo infinito?
3. Conclusión. ¿Hemos aprendido algo de todo esto?

